

Trabajo Práctico 2

TDA Diccionario e Implementaciones Dinámicas

Especificación, implementación dinámica, comparación estática/dinámica y utilización

Algoritmos y Estructuras de Datos II — Grupo 2

Lenguaje: Java

Integrantes: Alejo Maggi · Mateo Galli · Juan Ventola · Francisco Medina · Ayelén Balmaceda

Fecha: 21 / 09 /2026

Parte 0 — Preguntas Teóricas

1. ¿Qué diferencia hay entre el TDA Diccionario y el TDA Conjunto? ¿Por qué el Diccionario necesita `existeClave` antes de definir?

El Conjunto guarda elementos únicos y sin orden: lo único que importa es si un elemento está o no está (pertenece). El Diccionario guarda pares (clave, valor), donde las claves son únicas y cada una tiene asociado un valor, y se busca por clave para recuperar su valor. El Diccionario necesita `existeClave` antes de definir porque definir tiene doble comportamiento: si la clave no existe inserta un par nuevo, y si ya existe actualiza el valor sin duplicar la clave. Para decidir cuál de las dos cosas hacer —y así mantener la unicidad de las claves— primero hay que saber si la clave ya está.

2. ¿Qué es un Nodo en una implementación dinámica? ¿Qué información mínima debe guardar para encadenar los elementos entre sí?

Un Nodo es la unidad básica de una estructura dinámica: un objeto que guarda un dato y una referencia al siguiente nodo. La estructura se arma encadenando nodos, cada uno apuntando al que sigue. Para el Diccionario dinámico, lo mínimo que debe guardar es la clave, el valor y la referencia al siguiente nodo (siguiente). Sin esa referencia no se pueden encadenar los elementos; sin clave y valor no cumple su propósito.

3. ¿Por qué insertar al principio es $O(1)$ en una implementación dinámica y $O(n)$ en una estática (con arreglo)?

En la implementación dinámica, insertar al principio consiste en crear un nodo nuevo, apuntar su referencia siguiente a la antigua cabeza y mover la cabeza al nodo nuevo: son unos pocos pasos de cantidad fija, sin importar cuántos elementos haya, y no se toca ningún otro nodo, por eso es $O(1)$. En la estática con arreglo, la posición 0 ya está ocupada; para insertar al principio hay que

desplazar todos los elementos un lugar hacia la derecha para hacer espacio, y ese corrimiento es proporcional a la cantidad de elementos, por eso es $O(n)$.

4. Mencionen una ventaja y una desventaja de una implementación dinámica frente a una estática, en términos de uso de memoria.

Ventaja: usa exactamente la memoria que necesita, creciendo y decreciendo nodo a nodo, sin reservar de antemano un tamaño fijo; no hay lugares vacíos reservados ni un tope máximo prefijado (no se queda “llena” mientras haya memoria disponible). Desventaja: cada nodo gasta memoria extra (overhead) en la referencia al siguiente y en el propio objeto, que el arreglo no necesita; además los nodos quedan dispersos en memoria en lugar de estar contiguos.

5. Si dos implementaciones caen en la misma clase de complejidad teórica (mismo Big O), ¿por qué en la práctica pueden medir tiempos reales distintos? Mencionen un factor.

Porque el Big O describe cómo crece el costo al aumentar n , ignorando las constantes y los detalles de la máquina. Dos soluciones $O(n)$ pueden tener constantes muy distintas. Un factor concreto es la localidad de referencia: el arreglo guarda los datos contiguos, así que el procesador los trae a la caché de a bloques y los recorre rápido; la cadena de nodos los tiene dispersos, y seguir cada referencia siguiente puede implicar un salto a otra zona de memoria (fallo de caché), lo que la hace más lenta aunque el orden de crecimiento sea el mismo.

6. ¿Por qué la medición empírica de tiempos (medirNanos, con warmup y mejor tiempo de varias repeticiones) complementa, pero no reemplaza, al análisis de complejidad teórico (Big O)?

La medición empírica da el costo real en una máquina, con sus constantes y sus cachés: captura justo lo que el Big O ignora, y por eso sirve para distinguir dos soluciones del mismo orden de crecimiento. Pero no lo reemplaza porque una medición es puntual: depende del hardware, del estado de la máquina y del rango de n probado, no garantiza cómo va a escalar más allá de lo medido ni es necesariamente reproducible en otra máquina. El Big O aporta la tendencia de crecimiento válida en general e independiente de la máquina. Uno dice “cómo crece” (teórico, general) y el otro “cuánto tarda acá” (empírico, concreto); se necesitan los dos.

Parte 1 — Especificación del TDA

Diccionario

Dominio: Diccionario = colección de pares (clave, valor) en la que cada clave es única y tiene asociado exactamente un valor. No hay un orden particular entre los pares.

Operaciones

crear() → Diccionario

Post: devuelve un diccionario vacío (sin pares).

definir(d: Diccionario, clave, valor) → Diccionario

Post: clave queda asociada a valor en d. Si la clave no existía, se agrega el par (clave, valor); si ya existía, se actualiza su valor (la cantidad de claves no cambia en ese caso).

obtener(d: Diccionario, clave) → valor

Pre: existeClave(d, clave) — la clave pertenece a d.

Post: devuelve el valor asociado a clave en d, sin modificar d.

eliminar(d: Diccionario, clave) → Diccionario

Post: si clave pertenece a d, se elimina el par (clave, valor); si no pertenece, d queda sin cambios.

existeClave(d: Diccionario, clave) → boolean

Post: devuelve true si clave pertenece a d, false en caso contrario.

esVacio(d: Diccionario) → boolean

Post: devuelve true si d no tiene pares (cantidadClaves(d) = 0).

cantidadClaves(d: Diccionario) → entero

Post: devuelve la cantidad de claves (pares) que contiene d.

claves(d: Diccionario) → Lista

Post: devuelve una lista con todas las claves de d, sin un orden particular; d no se modifica.

Parte 2 — Implementación dinámica

Implementada como una cadena de `NodoDiccionario`. Los pares nuevos se insertan al principio de la cadena (sin mantener ningún orden). Se mantiene un contador de claves para que `cantidadClaves()` sea $O(1)$.

2.1 Análisis de complejidad

n = cantidad de claves del diccionario.

Operación	Complejidad	Justificación
<code>crear()</code>	$O(1)$	inicializa la cabeza en null y el contador en 0: pasos de cantidad fija.
<code>definir(clave, valor)</code>	$O(n)$	recorre la cadena (<code>buscarNodo</code>) para ver si la clave ya está y decidir si actualiza o inserta; la inserción al principio es $O(1)$, pero la búsqueda previa domina.
<code>obtener(clave)</code>	$O(n)$	recorre la cadena hasta encontrar la clave (o hasta el final); en el peor caso la clave está al final.
<code>eliminar(clave)</code>	$O(n)$	recorre la cadena buscando la clave para desengancharla.
<code>existeClave(clave)</code>	$O(n)$	recorre la cadena hasta hallar la clave o llegar al final.
<code>esVacio()</code>	$O(1)$	solo consulta si la cabeza es null.

cantidadClaves()	$O(1)$	devuelve el contador que se mantiene actualizado en cada definir/eliminar; sin el contador sería $O(n)$.
claves()	$O(n)$	recorre la cadena una vez, agregando cada clave a la Lista.

Nota: La mayoría de las operaciones son $O(n)$ porque comparten el mismo recorrido de la cadena para localizar una clave. Las que dan distinto (crear, esVacio, cantidadClaves) son $O(1)$ porque no dependen de recorrerla.

Parte 3 — Comparación estática vs. dinámica

La versión estática usa dos arreglos paralelos (claves y valores) de tamaño fijo, con la misma estrategia de búsqueda lineal que la dinámica; solo cambia el mecanismo de almacenamiento (arreglo vs. nodos).

3.1 Medición empírica

Tiempos reales de definir() (clave nueva) y obtener() (peor caso) medidos con medirNanos() (warmup + mejor tiempo de varias repeticiones).

n	definir() Estática	definir() Dinámica	obtener() Estática	obtener() Dinámica
1.000	4,50 μ s	6,80 μ s	15,60 μ s	7,50 μ s
10.000	8,90 μ s	22,00 μ s	8,30 μ s	21,40 μ s
100.000	136,70 μ s	741,10 μ s	70,90 μ s	755,60 μ s

Nota: Tiempos medidos en la máquina del grupo con la clase MedicionTiempos (mejor de 7 repeticiones, con warmup), expresados en microsegundos (μ s).

3.2 Conclusión

La tendencia medida coincide en lo esencial con la complejidad $O(n)$ de la Parte 2. Al pasar de $n = 10.000$ a $n = 100.000$ (diez veces más claves), los tiempos crecen en el mismo orden: por ejemplo, obtener() dinámico pasa de 21,40 a 755,60 μ s, y definir() dinámico de 22,00 a 741,10 μ s, un crecimiento consistente con lo lineal. Las dos implementaciones son del mismo orden $O(n)$, pero no miden lo mismo: la estática es más rápida y su ventaja se agranda con n . En $n = 100.000$ la diferencia es marcada: obtener() tarda 70,90 μ s en la estática contra 755,60 μ s en la dinámica, alrededor de diez veces menos.

La razón es la vista en la Clase 5: localidad de referencia y overhead de memoria. El arreglo guarda las claves en posiciones contiguas, así que al recorrerlo el procesador las trae a la caché de a bloques y las lee muy rápido; la cadena de nodos, en cambio, tiene los nodos dispersos por la memoria, y seguir cada referencia siguiente puede provocar un fallo de

caché (un salto a otra zona). Además, cada nodo agrega el overhead del objeto y de su referencia. Por eso, con el mismo orden de crecimiento, la versión con arreglo tiene una constante menor y gana en tiempo real.

Esto ilustra por qué la medición empírica complementa al Big O sin reemplazarlo: el Big O captura cómo escala el costo, pero no las constantes —como el efecto de la caché— que deciden cuál de dos soluciones del mismo orden es más rápida en la práctica.

Nota: En $n = 1.000$, obtener() estático ($15,60 \mu s$) aparece más alto que en $n = 10.000$ ($8,30 \mu s$), lo que no refleja un crecimiento real sino ruido de medición: es la primera operación medida y la JVM todavía está optimizando el código (warmup/JIT). Por eso la técnica toma el mejor de varias repeticiones y la tendencia se lee mejor en los n grandes, donde se estabiliza.

Parte 4 — Utilización

n = cantidad de claves de los diccionarios involucrados. Los métodos trabajan únicamente contra la interfaz del TDA, por lo que funcionan igual con las dos implementaciones.

Método	Complejidad	Justificación
combinarDiccionarios	$O(n^2)$	recorre las n claves de cada diccionario y, por cada una, hace obtener y definir, que son $O(n)$: $n \cdot O(n) = O(n^2)$.
invertir	$O(n^2)$	recorre las n claves y, por cada una, hace un obtener y un definir (ambos $O(n)$): $O(n^2)$.
contarValoresMayoresA	$O(n^2)$	recorre las n claves y, por cada una, hace un obtener $O(n)$: $O(n^2)$.
clavesOrdenadas	$O(n^2)$	obtener la lista de claves es $O(n)$; el ordenamiento simple (inserción) es $O(n^2)$ en el peor caso, que domina el costo.

Nota: Aunque cada operación del TDA es $O(n)$, al llamarla dentro de un bucle que recorre las n claves el costo total del método pasa a ser $O(n^2)$, como advierte el enunciado.